Data Science & AI

DATA STRUCTURES Through Python

Linked Lists

Lecture No.- 04

By- Satya sir

Recap of Previous Lecture

SLL - Deletion

- Delete first Node
- Delete last Node
- Delete middle Node

Topics to be Covered

Doubly Linked List

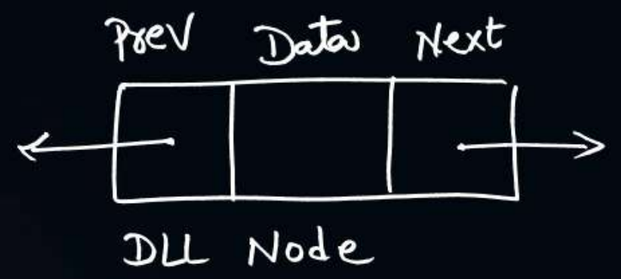
- Insertion
- Deletion
- Search
- Traverse

- Circular Linked List

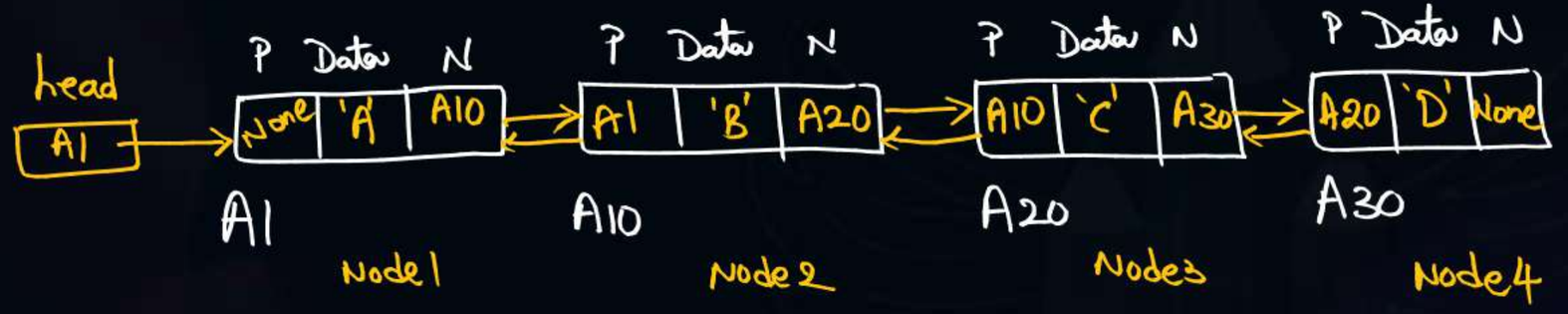
Topic : DLL – Insertion, Deletion

Doubly Linked Lists

DLL Node : 1) Data Part 2) Next Node Reference 3) Previous Node Reference



DLL with multiple Nodes :



Topic : DLL – Insertion, Deletion

Construct DLL Node

```
class DLL_Node :

    def __init__ (self, data):
        self.data = data
        self.prev = None
        self.next = None
```

```
class DLL_head :
    def __init__ (self):
        self.head = None
```

head

None

```
head = DLL_head( )
```

```
ob1 = DLL_Node('A')
```

P	Data	N
None	'A'	None

```
ob2 = DLL_Node('B')
```

P	Data	N
None	'B'	None

```
ob3 = DLL_Node('C')
```

P	D	N
None	'C'	None

Topic : DLL – Insertion, Deletion

Construct DLL with multiple Nodes ≅ Insertion at End of DLL without Tail

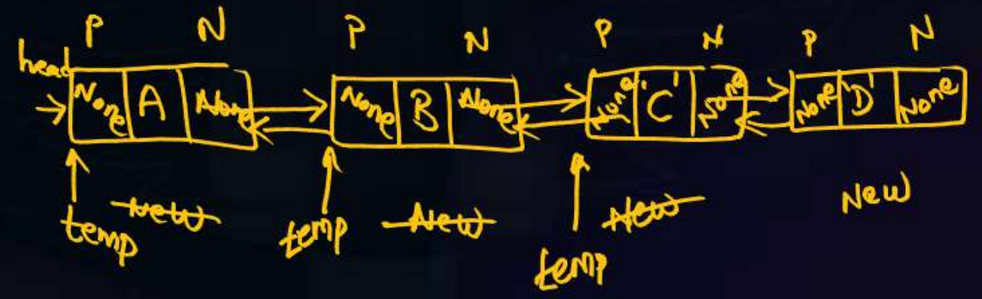
```
class DLL_Node:
    def __init__(self, data):
        self.data = data
        self.Prev = None
        self.Next = None

class DLL_head:
    def __init__(self):
        self.head = None
```

```
head = DLL_head( )
def construct_DLL (self, Value):   #def Insert_At_End
    New = DLL_Node (Value)
    if self.head is None :
        self.head = New
        New.Next = None
        New.Prev = None
    else:
        temp = Self.head
        while temp.next is not None:
            temp = temp.next
        temp.next = New
        New.Prev = temp
        New.Next = None
```

construct_DLL ('A') ✓
construct_DLL ('B') ✓
construct_DLL ('C') ✓
construct_DLL ('D') ✓

Time Complexity : $O(n)$



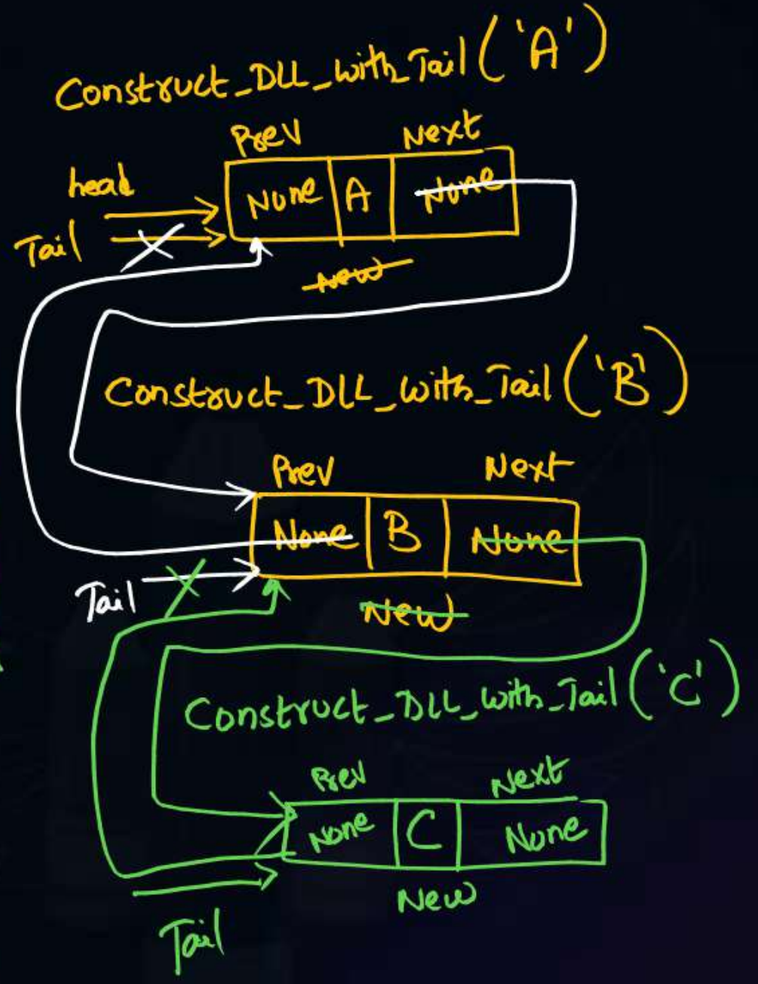
Topic : DLL – Insertion, Deletion

```
class DLL_head_Tail :
  def __init__(self):
      self.head = None
      self.Tail = None
```

Insertion_at_End | Construct_DLL with Tail at last Node

```
def Construct_DLL_with_Tail(self, Value):
    New = DLL_Node(value)
    if self.head is none : # Empty list
        self.head = New
        Self.Tail = New
        New.Prev = None
        New.next = None
    else :
        Tail.next = New
        New.Prev = Tail
        New.next = None
        Tail = New
```

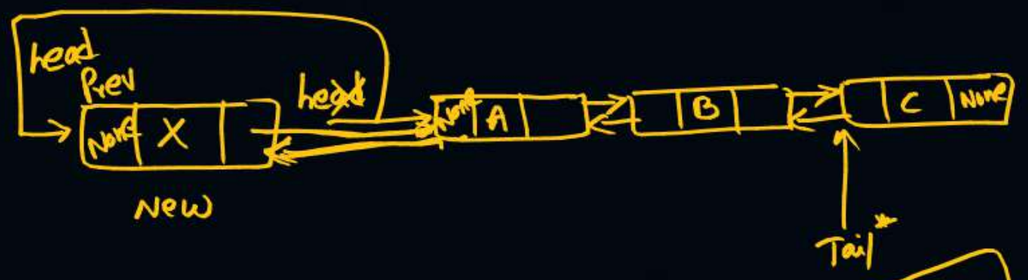
$\boxed{\text{Time Complexity : } O(1)}$



Topic : DLL – Insertion, Deletion

Insertion_at_beginning with or without Tail

```
def insert_At_front(self, value):
    New = DLL_Node(value)
    if self.head is None :
        self.head = New
        # self.Tail = New        if Tail is Considered
        New.Prev = None
        New.next = None
    else:
        New.next = self.head
        New.next.Prev = New # self.head.Prev = new
        New.Prev = None
        self.head = New
```



Time Complexity : O(1)

Topic : DLL – Insertion, Deletion

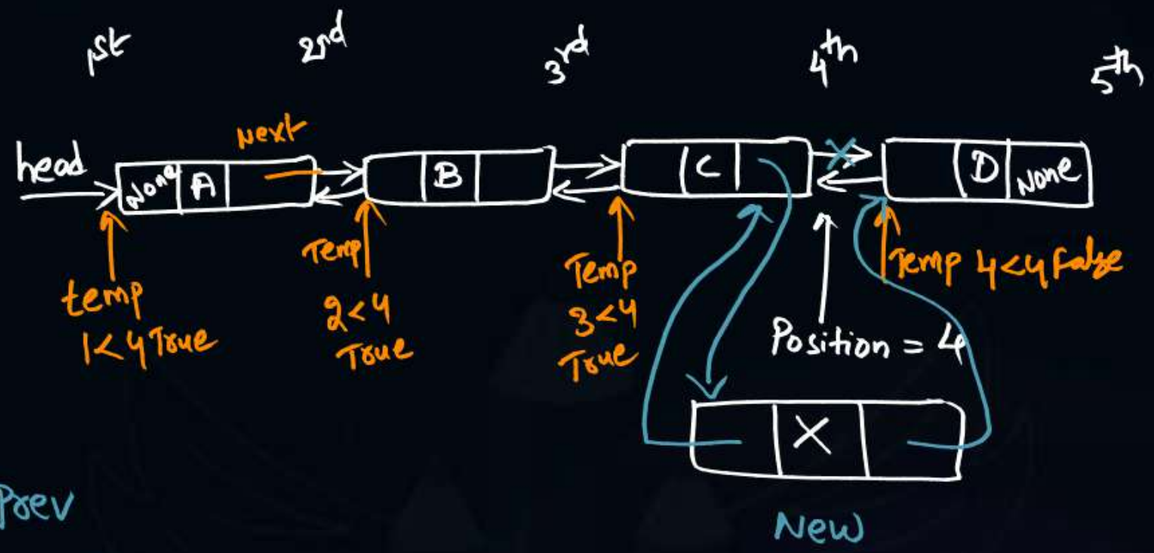
Insertion at given Position without Tail

Worst Case : Second last Position

Time Complexity : $O(n)$

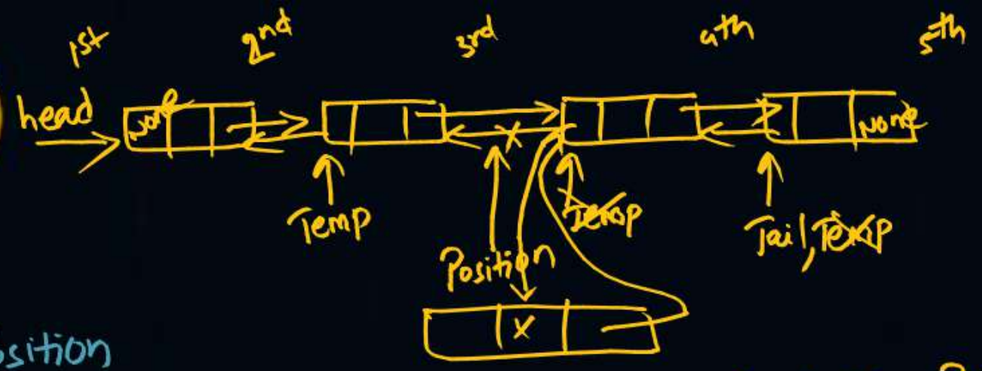
```
def Insert_DLL_middle ( self, value, Position )

    Count = 1
    New = DLL_Node (value)
    if self.head is None : #Empty list
        self.head = New
        New.Prev = None
        New.next = None
    else : temp = self.head
        while Count < Position :
            temp = temp.next
            Count = Count + 1
        #End of while
```




```
New.Prev = Temp.Prev
New.Prev.next = New #Temp.Prev.next = New
New.next = Temp
Temp.Prev = New # New.next.Prev = new
```

Topic : DLL – Insertion, Deletion



Insertion into DLL at middle with Tail

Worst Case : Insertion at Exactly middle Position

```
def insert_DLL_middle_fromhead(self, value, Position):
    Count = 1
    New = DLL_Node(value)
    if self.head is None:
        self.head = New
        self.Tail = New
        New.next = None
        New.Prev = None
    else: temp = self.head
        while Count < Position
            temp = temp.next
            Count = Count+1
        New.Prev = temp.Prev
        temp.Prev.next = New
        New.Next = temp
        temp.Prev = new
```

$T.C = O(n)$

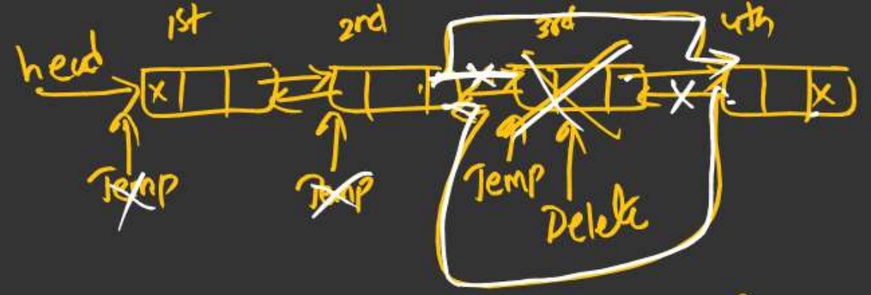
Time Complexity :

$$O(n/2) == O(n)$$

```
def insert_DLL_middle_fromTail(self, Value, Position, N):
                                          #3      #4
    Count = N+1   # 5
    New = DLL_Node(Value)
    if self.head is None:
        self.head = New
        self.Tail = New
        New.next = None
        New.Prev = None
    else:
        Temp = self.Tail
        while Count > Position:
            Temp = Temp.Prev
            Count = Count-1
```

```
New.Next = Temp.next
New.next.Prev = New
Temp.next = New
New.prev = Temp
```

DLL - Deletion of middle Node



Without Tail : Worst Case : 2nd last Node as middle Node

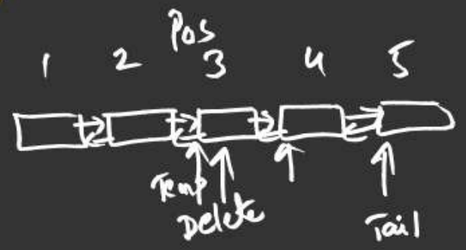
```
def Delete_middle_DLL (self, Position) : # Position==3
    count=1
    if self.head is None :
        Print('Empty list')
        return
    elif self.head.next is None: # only 1 Node
        Temp = self.head
        self.head = None
        Temp = None
    else:
        Temp = self.head
        while count < Position:
            Temp = Temp.next
            count = count+1
        # End of while
```

Time Complexity
$O(n)$

```
Temp.Prev.next = Temp.next
Temp.next.Prev = Temp.Prev
Temp = None
```

With Tail (Worst case : Deletion of Exactly middle Node)

```
def Deletion_DLL_with Tail (self, Pos):
    count = N+1
```




```
else: temp = self.Tail
    while count > Position
        Temp = Temp.prev
        count = count-1
```

Same here

Topic : CLL – Insertion, Deletion

Please go through

```
class Node:
  def __init__(self,data):
    self.data = data;
    self.next = None;

class CreateList:
  #Declaring head and tail pointer as null.
  def __init__(self):
    self.head = Node(None);
    self.tail = Node(None);
    self.head.next = self.tail;
    self.tail.next = self.head;
```

```
def addAtStart(self,data):
    newNode = Node(data);
    #Checks if the list is empty.
    if self.head.data is None:
      #If list is empty, both head and tail would point to new node.
      self.head = newNode;
      self.tail = newNode;
      newNode.next = self.head;
    else:
        #Store data into temporary node
        temp = self.head;
        #New node will point to temp as next node
        newNode.next = temp;
        #New node will be the head node
        self.head = newNode;
        #Since, it is circular linked list tail will point to head.
        self.tail.next = self.head;
```

Topic : CLL – Insertion, Deletion

```
def display(self):
   current = self.head;
   if self.head is None:
     print("List is empty");
     return;
   else:
       #Prints each node by incrementing pointer.
      print(current.data),
      while(current.next != self.head):
         current = current.next;
         print(current.data),
      print("\n");
```

```
class CircularLinkedList:
  cl = CreateList();

  cl.addAtStart(1);
  cl.display();
 cl.addAtStart(2);
  cl.display();
 cl.addAtStart(3);
  cl.display();
   cl.addAtStart(4);
  cl.display();
```

2 mins Summary

Doubly linked list

- Insertion
  - front
  - last
  - middle
- Deletion
  - middle Node
  - first node
  - last node

  Try self

To be contd ...

THANK - YOU